

DSA Day 43: Collision Resolution & Hashing

Book: Data Structures Through C in Depth - Ch9 §9.3.2-9.3.3 · pp 480-491 | Code in Python

Overview

Collisions are resolved by open addressing (linear/quadratic/double probing) or separate chaining (a list per bucket).

Key concepts to master

- probing sequences
- load factor
- separate chaining
- bucket hashing

Python implementation

```
class ChainingHash:
    def __init__(self, m): self.m=m; self.t=[] for _ in range(m)
    def put(self, k, v):
        b=self.t[hash(k)%self.m]
        for i, (kk, _) in enumerate(b):
            if kk==k: b[i]=(k, v); return
        b.append((k, v))
    def get(self, k):
        for kk, v in self.t[hash(k)%self.m]:
            if kk==k: return v
```

Complexity

Avg $O(1)$; worst $O(n)$ if all collide.

Common pitfalls

- High load factor slows probing
- Deletion in open addressing needs tombstones

Practice

- Compare probing vs chaining
- How does load factor affect speed?

